provider like OpalStack, whose dashboard includes installation modules that connect to their web server. On OpalStack, you would install three modules: Django and two file-serving modules (one for your static files and the other for uploads).

One downside of a VPS is it may be up to you to maintain your associated software. For example, if you've installed your own Apache web server, you are responsible for patching it. This comes back to the spectrum: a near-bare-metal Linode instance requires more work than an OpalStack account, where most of this is taken care of for you.

If your Django project uses a database, well, that's one more thing you have to worry about, isn't it? On the extreme left, you install and configure a database yourself. I generally recommend against this. Most VPS providers also offer cloud services, including databases. This is the better choice, as the database gets tuned by a professional and backups typically get handled for you.

If you have the good fortune of your site becoming very popular, you also have a new problem: scaling. With enough traffic, a single instance of a VPS is insufficient. Most VPS providers offer some load-balancing mechanism that splits traffic among different servers, but this requires more configuration.

### B.2.2 Platform as a service

We find PaaS providers on the right side of the deployment spectrum. These hosting companies abstract away all the details of deployment. They tend to be a little more expensive than VPS, but there is a lot less work, in both getting them going and maintaining them. DigitalOcean has a separate offering from their Droplets VPS, called the App Platform; this and Heroku are common PaaS choices. The interface to Heroku, for example, is a GitHub repository. You point your Heroku account at your repo, and when you update the main branch, Heroku automatically deploys. Their system understands Django intimately and knows exactly where the different kinds of files go and how to host them. You can even include a configuration file in your repo to fine-tune your deployment options.

There is also the container approach, using Docker or Kubernetes. If you're new to Django and the web world, I wouldn't recommend going down this path, unless you are already very familiar with containers. Containers can help you scale massively, but most sites don't need them to get going. There is a fairly steep learning curve to using containers, and if you're already experimenting with your first Django deployment, adding containers on top of that makes for a difficult hill to climb. If you are keen on Docker, the PaaS host Fly.io might be the right choice for you.

The previous paragraph can more or less be repeated, replacing the word *container* with the word *serverless*. Serverless configurations, like AWS Lambda, can provide massive scaling options at the cost of warm-up time (how long it takes to start a new instance in the background), but a lot of moving parts are needed to get Django going on Lambda. You'll need an S3 instance for storing your files, an RDS database, Lambda itself, and several other services to wire all this together. As with containers, if you aren't already intimately familiar with AWS, this isn't the place to start.